

## CONTEO DE OBJETOS SOBRE VIDEO:

```
import cv2
from ultralytics import YOLO
import sys

# 1. Cargar modelo
model = YOLO("yolo11n.pt")

# 2. Abrir el video de entrada
video_path = "video_trafico.mp4"
cap = cv2.VideoCapture(video_path)
if not cap.isOpened():
    print("Error: No se pudo abrir el video.")
    sys.exit()

# Configurar para guardar el video de salida
ancho = int(cap.get(cv2.CAP_PROP_FRAME_WIDTH))
alto = int(cap.get(cv2.CAP_PROP_FRAME_HEIGHT))
fps = int(cap.get(cv2.CAP_PROP_FPS))
writer = cv2.VideoWriter('trafficCam_resultado.mp4',
    | | | | | | | cv2.VideoWriter_fourcc(*'mp4v'), fps, (ancho, alto))

# --- VARIABLES PARA EL CONTEO ---
ids_contados = set()
# Diccionario: guarda qué clase tiene cada ID (ID → clase)
id_a_clase = {}

print("Procesando video con seguimiento de IDs...")
```

Definimos el modelo de YOLO, la ruta del video que vamos a usar y configuramos las variables para guardar el video de salida.

```

# 3. Bucle frame a frame
frame_count = 0
while cap.isOpened():
    success, frame = cap.read()
    if success:
        # --- APLICAR TRACKING (Seguimiento) ---
        results = model.track(frame, conf=0.5, classes=[2, 3, 5, 7], persist=True, verbose=False)

        # Comprobar si hay detecciones con ID
        if results[0].boxes.id is not None:
            ids = results[0].boxes.id.int().cpu().tolist()
            clases = results[0].boxes.cls.int().cpu().tolist()

            for id_vehiculo, cls_id in zip(ids, clases):
                ids_contados.add(id_vehiculo)
                # Guardar la clase asociada a cada ID
                id_a_clase[id_vehiculo] = model.names[cls_id]

        # --- VISUALIZAR ---
        frame_anotado = results[0].plot()

        # Dibujar el contador actual en el video (esquina superior izquierda)
        cv2.putText(frame_anotado, f"Vehiculos totales: {len(ids_contados)}",
                    (50, 50), cv2.FONT_HERSHEY_SIMPLEX, 1, (0, 255, 0), 2)

        # --- GUARDAR EN ARCHIVO ---
        writer.write(frame_anotado)

        frame_count += 1
        if frame_count % 30 == 0:
            print(f"Frames procesados: {frame_count} | Vehículos detectados: {len(ids_contados)}", end="\r")
    else:
        break

```

Usamos un bucle para recorrer frame a frame, aplicando una serie de parámetros como la confianza y los tipos de clases. Vamos mostrando el número total de vehículos que le modelo está siendo capaz de visualizar.

```

# 4. Limpiar memoria
cap.release()
writer.release()

# --- CONTAR CUÁNTOS DE CADA CLASE ---
conteo_por_clase = {}
for clase in id_a_clase.values():
    conteo_por_clase[clase] = conteo_por_clase.get(clase, 0) + 1

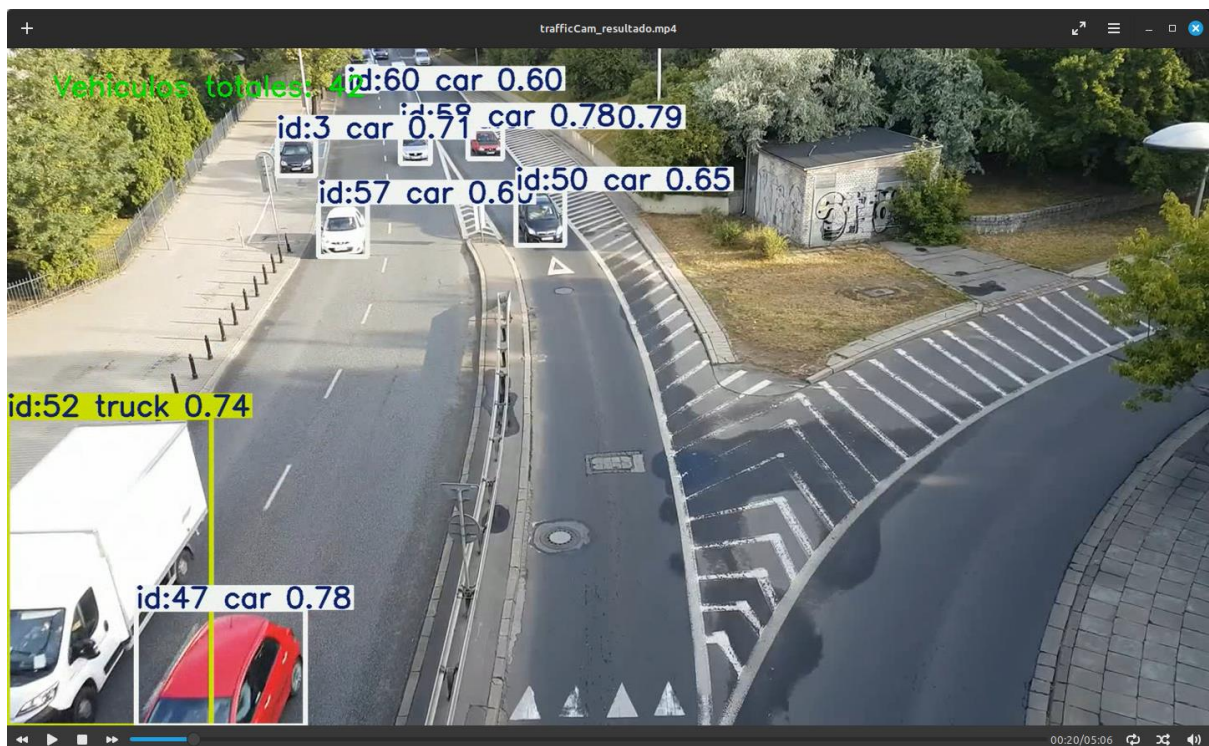
print(f"\n\nProceso finalizado con éxito.")
print(f"-----")
print(f"TOTAL DE VEHÍCULOS ÚNICOS: {len(ids_contados)}")
print(f"-----")
for clase, cantidad in conteo_por_clase.items():
    print(f"  {clase}: {cantidad}")
print(f"-----")
print(f"El video se ha guardado en: trafficCam_resultado.mp4")

```

Limpiamos la memoria, contamos cuantos vehículos de cada clase han sido detectados. Al terminar el bucle mostramos la cantidad de vehículos detectados y cuantos de cada clase.

```
Procesando video con seguimiento de IDs...  
Frames procesados: 9180 | Vehículos detectados: 546  
  
Proceso finalizado con éxito.  
-----  
TOTAL DE VEHÍCULOS ÚNICOS: 546  
-----  
car: 494  
truck: 31  
bus: 21  
-----  
El video se ha guardado en: trafficCam_resultado.mp4
```

Mostramos por pantalla todos los resultados finales.



Esta sería una captura del video de salida donde podemos ver la detección de los vehículos y el grado de confianza que tiene que cada uno.